

VC Memory AI

An AI memory layer for lean VC teams.

Dibyajyoti Behera

■ PROBLEM & WHO IT'S FOR

A lean VC team meets hundreds of startups a year. Every deck, call, memo, and rejection creates real knowledge — and most of it is **lost**, scattered across drives, inboxes, and people's heads.

The cost shows up when a founder you passed on returns a year later: *did we meet them, what did we conclude, why did we pass?* Answering means digging through old files or hoping someone remembers.

It's built for **lean investment teams** — a handful of people evaluating deals, managing the portfolio, and running operations, with no one whose job is to be the firm's memory. When that context slips, the team repeats work and the firm **stops compounding its own learning**.

■ WHAT I BUILT

A **memory layer** that turns scattered documents into knowledge the firm can reuse.

BEFORE

A founder returns after a year → the team searches scattered notes, Slack, and memory.

AFTER

Ask VC Memory AI → the past context, the decision, and the reasoning come back instantly, with citations.

- **Recall** — ask the firm's history in plain language; get grounded, **cited** answers.
- **Compare** — drop in a new deck; see the closest past deals and **what the firm concluded then**.
- **Prepare** — generate a partner-ready brief for any company in seconds.

It **supports judgment** — surfacing context, risks, and sharper questions. It never predicts winners or makes the invest/pass call.

■ TOOLS & MODELS — AND WHY

- **Llama 3.3 70B (via Groq)** — reasoning quality mattered for extracting nuanced investment context: decisions, concerns, and the *why* behind a pass. Groq's free tier ran it at **\$0 with no card**, and the model sits behind a thin interface, so moving to Claude or GPT later is a one-line change.
- **Voyage embeddings + retrieval (RAG)** — so every answer is **grounded in the firm's real records and cited**, not invented by the model.
- **Next.js + Postgres (Neon), on Vercel** — one full-stack codebase that actually deploys and persists its memory.

■ WHAT DIDN'T WORK — AND HOW THE IDEA EVOLVED

Pitch-deck analyzer → "that only solves one moment" → the real problem is lost institutional memory → **a memory layer where knowledge compounds**

My first build was a **pitch-deck analyzer**. It worked, but felt generic — any investor can already paste a deck into ChatGPT. The realization: the value isn't understanding one company faster, it's **combining a new deal with everything the firm already knows**. That reframed the product from "summarize this startup" to "help me understand it using what we already know."

The hard parts were **infrastructure, not the AI**: data vanished on serverless until I moved to a persistent database; free-tier limits forced batching and graceful handling. I also tried a cheaper model for chat to save budget — it **blurred decision states** (called a *tracking deal passed*), so I reverted. Correctness on the memory matters more than saving tokens.

■ WHAT I'D EXTEND

- **Automatic capture** from Gmail, Slack, Notion, and Drive — so memory builds itself instead of by manual upload.
- **Portfolio relationship tracking** — surface "who haven't we spoken to in 90 days" and open follow-ups.
- **Founder follow-up intelligence** — read portfolio updates for risks and concrete ways to help.
- **CRM sync (e.g. Affinity)** — so the memory lives where the team already works.

A lean team's knowledge should compound over time — this is the layer that makes that happen.